1. **LOGIN**

1. **Introduction**

   This use case allows a user (Student, Faculty, or Admin) to log into the system by providing valid credentials.

2. **Actors**

   1. Student
   2. Faculty
   3. Admin

3. **Pre-Conditions**

   1. The user must be registered in the system.
   2. The system must be online and accessible.

4. **Post-Conditions**

   1. If login is successful, the user is directed to their respective dashboard.
   2. If login fails, an error message is shown, and the attempt is recorded.

5. **Flow of Events**

   5.1 Basic Flow

   1. The user enters a username and password.
   2. The system validates the credentials.
   3. If valid, the user is redirected to their dashboard.

   5.2 Alternative Flow

   1. If credentials are incorrect, an error message is displayed.
   2. If the account is locked due to multiple failed attempts, the user is asked to reset the password.

6. **Special Requirements**

   None

7. **Use Case Relationships**

   None

2. **STUDENT REGISTRATION**

1. **Introduction**

   This use case allows a new student to register in the UMS by providing personal and academic details.

2. **Actors**

1. Student
2. Admin

3. **Pre-Conditions**

The student must provide valid documents (ID proof, academic certificates).

4. **Post-Conditions**

1. A student record is created in the database.
2. A unique student ID is generated.

5. **Flow of Events**

5.1 Basic Flow

1. The student fills in the registration form.
2. The system verifies the entered details.
3. The system generates a unique student ID.
4. The system stores the student record.
5. Registration confirmation is sent to the student.

5.2 Alternative Flow

1. If any mandatory field is missing, an error is displayed.
2. If document verification fails, the registration is rejected.

6. **Special Requirements**

The system should handle duplicate entries gracefully.

7. **Use Case Relationships**

Associated Use Cases: Maintain Student Profile, Course Registration

3. MAINTAIN STUDENT PROFILE

1. **Introduction**

This use case allows students to update their profile details, such as phone number, address, and email.

2. **Actors**

1. Student
2. Admin

3. **Pre-Conditions**

The student must be logged in.

4. **Post-Conditions**

The student profile is updated in the database.

5. **Flow of Events**

5.1 Basic Flow

1. The student selects the "Edit Profile" option.
2. The system retrieves the existing profile data.
3. The student updates the details.
4. The system validates and saves the changes.
5. A confirmation message is displayed.

5.2 Alternative Flow

1. If validation fails (e.g., incorrect phone number format), an error message is displayed.

6. **Special Requirements**

Profile changes should be logged for security purposes.

7. **Use Case Relationships**

Associated Use Cases: Student Registration, Login

4. SCHEDULE EXAMS

1. **Introduction**

This use case allows the admin to schedule exams for various courses.

2. **Actors**

1. Admin
2. Faculty

3. **Pre-Conditions**

The course and faculty must be assigned.

4. **Post-Conditions**

The exam schedule is saved and visible to students.

5. **Flow of Events**

5.1 Basic Flow

1. The admin selects the course and exam date.
2. The system validates the schedule.
3. The exam schedule is stored.
4. Students and faculty are notified.

5.2 Alternative Flow

1. If the selected date conflicts with another exam, an error message is displayed.

6. **Special Requirements**

Exams must not overlap for the same student.

7. **Use Case Relationships**

Associated Use Cases: Enter Marks, View Results

5. ENTER MARKS

1. **Introduction**

This use case allows faculty to enter student exam marks.

2. **Actors**

1. Faculty

3. **Pre-Conditions**

The student must have attended the exam.

4. **Post-Conditions**

Marks are stored in the database.

5. **Flow of Events**

5.1 Basic Flow

1. The faculty selects the course and student.
2. The faculty enters the marks.
3. The system validates the input.
4. Marks are saved in the system.

5.2 Alternative Flow

1. If the entered marks exceed the maximum limit, an error is displayed.

6. **Special Requirements**

Marks entry must be restricted to authorized faculty.

7. **Use Case Relationships**

Associated Use Cases: View Results

6. VIEW RESULTS

1. **Introduction**

This use case allows students to view their exam results.

2. **Actors**

1. Student

2. Faculty
3. Admin

3. **Pre-Conditions**

The marks must be entered into the system.

4. **Post-Conditions**

The student can see their results.

5. **Flow of Events**

5.1 Basic Flow

1. The student selects the "View Results" option.
2. The system retrieves the exam scores.
3. The results are displayed.

5.2 Alternative Flow

1. If the results are not yet published, a message is displayed.

6. **Special Requirements**

The system should ensure result privacy.

7. **Use Case Relationships**

Associated Use Cases: Enter Marks, Schedule Exams

7. ATTENDANCE TRACKING

1. **Introduction**

This use case allows faculty to track and manage student attendance records.

2. **Actors**

1. Faculty
2. Student
3. Admin

3. **Pre-Conditions**

1. The student must be enrolled in a course.
2. The faculty must be assigned to the course.

4. **Post-Conditions**

1. Attendance is recorded and stored in the database.
2. Students can view their attendance status.

5. **Flow of Events**

5.1 Basic Flow

1. Faculty selects the course and date.
2. Faculty marks attendance for students.
3. The system validates and stores the records.
4. Students receive attendance status.

5.2 Alternative Flow

1. If a student is marked absent incorrectly, the faculty can update the record.

6. **Special Requirements**

The system should prevent duplicate attendance entries.

7. **Use Case Relationships**

Associated Use Cases: View Results, Course Registration

8. MAINTAIN FACULTY PROFILE

1. **Introduction**

This use case allows faculty members to update and manage their personal and professional details.

2. **Actors**

1. Faculty
2. Admin

3. **Pre-Conditions**

The faculty must be registered in the system.

4. **Post-Conditions**

Faculty profile is updated and stored in the system.

5. **Flow of Events**

5.1 Basic Flow

1. Faculty logs into the system.
2. Faculty selects "Edit Profile" option.
3. Faculty updates details.
4. The system saves changes.
5. Confirmation is displayed.

5.2 Alternative Flow

1. If incorrect data is entered, the system displays an error message.

6. **Special Requirements**

Changes should be logged for security purposes.

7. **Use Case Relationships**

Associated Use Cases: Login, Course Registration

9. COURSE REGISTRATION

1. **Introduction**

This use case allows students to register for available courses.

2. **Actors**

1. Student
2. Admin

3. **Pre-Conditions**

1. The student must be registered in the system.
2. The course must be available.

4. **Post-Conditions**

1. The student is enrolled in the course.

5. **Flow of Events**

5.1 Basic Flow

1. The student selects the "Course Registration" option.
2. The system displays available courses.
3. The student selects courses.
4. The system verifies prerequisites.
5. The student is enrolled.
6. Confirmation is sent.

5.2 Alternative Flow

1. If prerequisites are not met, registration is denied.

6. **Special Requirements**

The system should prevent duplicate registrations.

7. **Use Case Relationships**

Associated Use Cases: Maintain Student Profile

10. MAINTAIN BOOK DETAILS

1. **Introduction**

This use case allows the librarian to update book records in the system.

2. **Actors**

1. Admin
2. Librarian

3. **Pre-Conditions**

The book must exist in the library database.

4. **Post-Conditions**

Book details are updated in the system.

5. **Flow of Events**

5.1 Basic Flow

1. Librarian logs in.
2. Librarian selects "Manage Books" option.
3. Updates book details.
4. The system validates and stores updates.

5.2 Alternative Flow

1. If invalid data is entered, an error is displayed.

6. **Special Requirements**

Updates should be logged.

7. **Use Case Relationships**

Associated Use Cases: Issue Book, Return Book

11. ISSUE BOOK

1. **Introduction**

This use case allows a student or faculty member to borrow a book from the university library.

2. **Actors**

1. Student
2. Faculty
3. Librarian

3. **Pre-Conditions**

1. The user must have a valid library membership.
2. The book must be available in stock.

4. **Post-Conditions**

1. The book is marked as issued in the library system.
2. The due date for returning the book is set.

5. **Flow of Events**

5.1 Basic Flow

1. The user selects a book to borrow.
2. The librarian scans the book and verifies user details.
3. The system updates the book status to "Issued" and sets a due date.
4. A confirmation receipt is generated.

5.2 Alternative Flow

1. If the book is not available, an error message is displayed.
2. If the user has exceeded the borrowing limit, issuance is denied.

6. **Special Requirements**

Integration with fine calculation for overdue books.

7. **Use Case Relationships**

Associated Use Cases: Return Book, Maintain Book Details

12. **RETURN BOOK**

1. **Introduction**

This use case allows users to return borrowed books to the library.

2. **Actors**

1. Student
2. Faculty
3. Librarian

3. **Pre-Conditions**

The book must be issued to the user.

4. **Post-Conditions**

1. The book is marked as available in the library system.
2. Late fees are calculated if applicable.

5. **Flow of Events**

5.1 Basic Flow

1. The user submits the book to the librarian.
2. The librarian scans the book and verifies details.

3. The system updates the book status to "Available".
4. If applicable, late fees are calculated and recorded.

5.2 Alternative Flow

1. If the book is damaged, an additional fine is imposed.

6. **Special Requirements**

Notification system for overdue books.

7. **Use Case Relationships**

Associated Use Cases: Issue Book

13. APPLY FOR HOSTEL

1. **Introduction**

This use case allows students to apply for hostel accommodation.

2. **Actors**

1. Student
2. Admin

3. **Pre-Conditions**

1. The student must be enrolled in the university.
2. Rooms must be available.

4. **Post-Conditions**

1. The student's application is recorded.
2. The system assigns a room or places the student on a waiting list.

5. **Flow of Events**

5.1 Basic Flow

1. The student submits a hostel application.
2. The admin verifies the request.
3. If a room is available, it is allocated.
4. Confirmation is sent to the student.

5.2 Alternative Flow

1. If no rooms are available, the student is put on a waiting list.

6. **Special Requirements**

Online application tracking system.

7. **Use Case Relationships**

Associated Use Cases: Allocate Room, Make Payment

14. **ALLOCATE ROOM**

1. **Introduction**

This use case allows an admin to allocate hostel rooms to students.

2. **Actors**

Admin

3. **Pre-Conditions**

1. The student must have applied for hostel accommodation.
2. Rooms must be available.

4. **Post-Conditions**

1. The room is assigned to the student.
2. The student record is updated.

5. **Flow of Events**

5.1 Basic Flow

1. The admin selects a student from the hostel application list.
2. The system checks room availability.
3. A room is assigned, and the student is notified.

5.2 Alternative Flow

1. If no rooms are available, the student remains on the waiting list.

6. **Special Requirements**

Automated room assignment system.

7. **Use Case Relationships**

Associated Use Cases: Apply for Hostel

15. **REGISTER FOR PLACEMENT**

1. **Introduction**

This use case allows students to register for placement opportunities.

2. **Actors**

1. Student
2. Placement Officer

3. **Pre-Conditions**

The student must be in the final year.

4. **Post-Conditions**

The student is registered for placement drives.

5. **Flow of Events**

5.1 Basic Flow

1. The student submits a placement registration request.
2. The placement officer verifies details.
3. The student is added to the placement list.

5.2 Alternative Flow

1. If registration requirements are not met, the request is rejected.

6. **Special Requirements**

Integration with company hiring portals.

7. **Use Case Relationships**

Associated Use Cases: Generate Report

16. MAKE PAYMENT

1. **Introduction**

This use case allows students to make payments for tuition, hostel, or library dues.

2. **Actors**

1. Student
2. Admin

3. **Pre-Conditions**

The student must have pending dues.

4. **Post-Conditions**

The payment is recorded, and receipts are generated.

5. **Flow of Events**

5.1 Basic Flow

1. The student selects the type of payment.
2. The system processes the payment.
3. A receipt is generated.

5.2 Alternative Flow

1. If payment fails, an error message is displayed.

6. **Special Requirements**

Secure payment gateway integration.

7. **Use Case Relationships**

Associated Use Cases: Check Pending Dues

17. **CHECK PENDING DUES**

1. **Introduction**

This use case allows students to check their pending dues.

2. **Actors**

Student

3. **Pre-Conditions**

The student must have financial records in the system.

4. **Post-Conditions**

A report of pending dues is displayed.

5. **Flow of Events**

5.1 Basic Flow

1. The student selects "Check Pending Dues".
2. The system retrieves pending payments.
3. The dues summary is displayed.

6. **Special Requirements**

Integration with payment history.

7. **Use Case Relationships**

Associated Use Cases: Make Payment

18. GENERATE REPORT

1. **Introduction**

This use case allows admins to generate various academic and financial reports.

2. **Actors**

Admin

3. **Pre-Conditions**

Data must be available in the system.

4. **Post-Conditions**

A report is generated.

5. **Flow of Events**

5.1 Basic Flow

1. The admin selects report type.
2. The system retrieves data and generates the report.

6. **Special Requirements**

7. **Use Case Relationships**

Associated Use Cases: View Results, Make Payment